



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Carandia Lorenzo Brenda Fernanda

N° de Cuenta: 319018961

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 26 de abril de 2025

CALIFICACIÓN: _____

Animación

Actividades realizadas

1. Separar del arco la parte del letrero
2. Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHFC Feria animado desplazándose las letras de derecha a izquierda como si fuera letrero LCD/LED de forma cíclica

Bloque de Código

```
if (tiempoNumeroCambiante >= 2.0f / velocidadNumeroCambiante) {
    // Incrementa el offset de textura en el eje U (desplazamiento horizontal)
    toffsetnumrocambiau += 0.23f;
    if (toffsetnumrocambiau > 1.0f)
        toffsetnumrocambiau = 0.07f;
    tiempoNumeroCambiante = 0.0f;
}

toffsetnumrov = 0.07;
toffset = glm::vec2(toffsetnumrocambiau, toffsetnumrov);

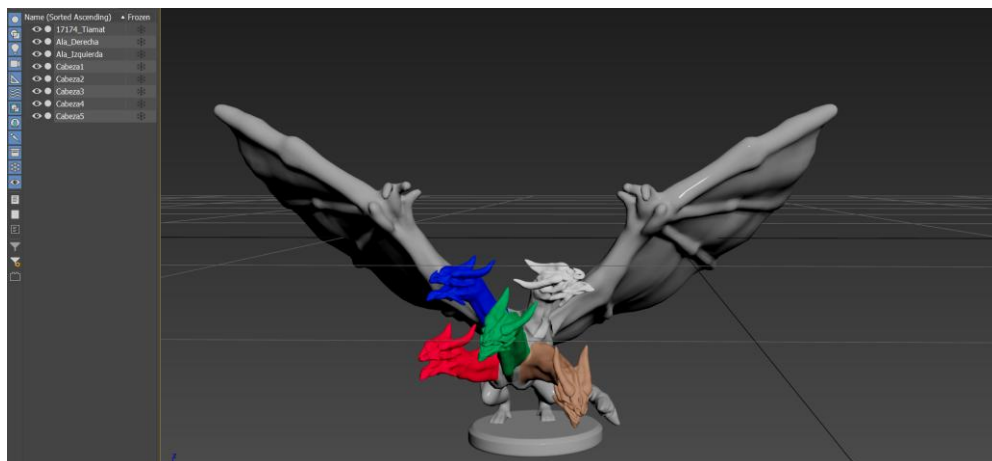
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(18.5f, 10.3f, 3.4f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(5.0f, 1.5f, 1.5f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Tipografia.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();
```

Le pasamos cada cuando va a aumentar en pixeles de nuestra textura de 1024x1024 de nuestra Tipografía para que lo vaya deslizando, además separamos nuestro modelo de puerta con el letrero.

3. Separar las cabezas (con todo y cuello) del Dragón y agregar las siguientes animaciones:

- Movimiento del cuerpo ida y vuelta.
- Aleteo
- Cada cabeza se mueve de forma diferente de acuerdo a una función/algorithmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc.)
- Cada cabeza debe de verse de un color diferente: roja, azul, verde, blanco, café.

Separamos las cabezas y las alas del cuerpo del dragón, además de que lo texturizamos desde 3d max.



E importamos los modelos y texturas a nuestro código

```
Model Kitt_M;  
Model Llanta_M;  
Model Dragon_M;  
Model Cabeza1;  
Model Cabeza2;  
Model Cabeza3;  
Model Cabeza4;  
Model Cabeza5;  
Model Tiamat_M;  
Skybox skybox;  
Model Ala_izquierda;  
Model Ala_derecha;  
Model Puerta;  
Model Letrero;
```

Bloque de código

```
// Lógica de movimiento y rotación del dragón  
if (avanza) {  
    if (movDragon > -300.0f) {  
        movDragon -= movOffset * deltaTime;  
        rotllanta += rotllantaOffset * deltaTime;  
  
        // Rotación gradual hacia adelante (0 grados)  
        if (rotacionDragonY > 0.0f) {  
            rotacionDragonY -= velocidadRotacion * deltaTime;  
            if (rotacionDragonY < 0.0f) rotacionDragonY = 0.0f;  
        }  
    }  
    else {  
        avanza = false;  
        // Iniciar rotación para mirar hacia atrás (180 grados)  
        rotacionDragonY = 180.0f;  
    }  
}  
else {  
    if (movDragon < 300.0f) {  
        movDragon += movOffset * deltaTime;  
        rotllanta -= rotllantaOffset * deltaTime;  
  
        // Rotación gradual hacia atrás (180 grados)  
        if (rotacionDragonY < 180.0f) {  
            rotacionDragonY += velocidadRotacion * deltaTime;  
            if (rotacionDragonY > 180.0f) rotacionDragonY = 180.0f;  
        }  
    }  
    else {  
        avanza = true;  
        // Iniciar rotación para mirar hacia adelante  
        rotacionDragonY = 0.0f;  
    }  
}
```

Para poder hacer que nuestro dragón avance utilizamos un if que dirá que alcanzado (300) el límite del piso regresará para el otro extremo, y así sucesivamente; además de que agregamos rotación para que gire cuando llegue al límite y no regrese de reversa.

Aleto

```

float angulo_aleteo = sin(glm::radians(angulovaria * 10.0f)) * 2.0f;
// Dragon
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(movDragon, 5.0f + 3 * sin(glm::radians(angulovaria * 1.0)), 6.0f));
model = glm::rotate(model, glm::radians(rotacionDragonY), glm::vec3(0.0f, 1.0f, 0.0f)); // Rotación en Y
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));

Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dragon_M.RenderModel();

//Ala derecha
model= modelaux;
model = glm::translate(model, glm::vec3(-0.688f, 5.754f, 1.741f ));
model = glm::rotate(model, glm::radians(angulo_aleteo), glm::vec3(1.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.688f,- 5.754f,- 1.741f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala_derecha.RenderModel();

//Ala izquierda
model = glm::translate(model, glm::vec3(-0.551f, 5.186f, -2.691f));
model = glm::rotate(model, glm::radians(angulo_aleteo), glm::vec3(1.0f, 1.0f, .0f));
model = glm::translate(model, glm::vec3(0.551f, -5.186f, 2.691f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala_izquierda.RenderModel();

```

Hacemos uso de una matriz modelaux, que nos servirá para la jerarquía, con todos nuestros elementos. Para el aleteo usamos la función sen para poder simularlo de una manera suave, de arriba hacia abajo.

Cabezas

```

// Constantes de control de velocidad
float VELOCIDAD_GLOBAL_CABEZAS = 0.5f; // Factor para reducir/incrementar TODOS los movimientos
float VELOCIDAD_CABEZAS1 = 0.1f;
float VELOCIDAD_CABEZAS2 = 0.1f;
float VELOCIDAD_CABEZAS3 = 0.1f;
float VELOCIDAD_CABEZAS4 = 0.1f;
float VELOCIDAD_CABEZAS5 = 0.1f;
float tiempoCabezas = 0.0f; // Acumulador de tiempo para animaciones

```

Declaramos variables globales que nos ayudaran a ajustar la velocidad para nuestras cabezas y lo podamos apreciar, así como para el tiempo de cada una de la animación.

```

// Actualizar tiempos de animación
tiempoNumeroCambiante += deltaTime;
tiempoCambioTextura += deltaTime;
tiempoCabezas += deltaTime * VELOCIDAD_GLOBAL_CABEZAS;

```

Actualizamos los tiempos de animación en cada ciclo

```

// CABEZA 1: Movimiento senoidal (más lento)
model = modelaux;
float amplitudSenoidal = 0.1f;
float offsetX1 = amplitudSenoidal * sin(VELOCIDAD_CABEZAS1 * tiempoCabezas * 2.0f); // Más lento
float offsetZ1 = amplitudSenoidal * cos(VELOCIDAD_CABEZAS1 * tiempoCabezas * 2.0f);
model = glm::translate(model, glm::vec3(offsetX1, 0.0f, offsetZ1));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
modelCabezas[0] = model; // Para cabeza 1
Cabeza1.RenderModel();

// CABEZA 2: Espiral de Arquímedes (más controlada)
model = modelaux;
float radioEspiral = 0.05f * (tiempoCabezas * 0.3f * VELOCIDAD_CABEZAS2); // Más lento
float anguloEspiral = tiempoCabezas * 1.0f * VELOCIDAD_CABEZAS2; // Más lento
float offsetX2 = radioEspiral * cos(anguloEspiral);
float offsetY2 = radioEspiral * sin(anguloEspiral);
model = glm::translate(model, glm::vec3(offsetX2, offsetY2, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
modelCabezas[1] = model; // Para cabeza 1
Cabeza2.RenderModel();

// CABEZA 3: Lemniscata (más lenta y suave)
model = modelaux;
float aLemniscata = 0.4f;
float tLemniscata = tiempoCabezas * 1.0f * VELOCIDAD_CABEZAS3; // Más lento
float denomLemniscata = 1.0f + sin(tLemniscata) * sin(tLemniscata);
float offsetX3 = (aLemniscata * cos(tLemniscata)) / denomLemniscata;
float offsetZ3 = (aLemniscata * sin(tLemniscata) * cos(tLemniscata)) / denomLemniscata;
model = glm::translate(model, glm::vec3(offsetX3, 0.0f, offsetZ3));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
modelCabezas[2] = model; // Para cabeza 1
Cabeza3.RenderModel();

// CABEZA 4: Movimiento circular más lento
model = modelaux;
float radioCircular = 0.05f;
float offsetX4 = radioCircular * sin(tiempoCabezas * 0.8f * VELOCIDAD_CABEZAS4);
float offsetY4 = radioCircular * cos(tiempoCabezas * 0.8f * VELOCIDAD_CABEZAS4) * 0.3f;
model = glm::translate(model, glm::vec3(offsetX4, offsetY4, 0.0f));
model = glm::rotate(model, sin(tiempoCabezas * 1.0f * VELOCIDAD_CABEZAS4) * 0.1f, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
modelCabezas[3] = model; // Para cabeza 1
Cabeza4.RenderModel();

// CABEZA 5: Movimiento vertical unido al cuerpo
model = modelaux;
float alturaCuello = -6.0f;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, alturaCuello));
float anguloVertical = sin(tiempoCabezas * VELOCIDAD_CABEZAS5) * 0.15f;
model = glm::rotate(model, anguloVertical, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -alturaCuello));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
modelCabezas[4] = model;
Cabeza5.RenderModel();

```

Dibujamos cada cabeza que será hija del cuerpo del dragón, además ya van a estar texturizadas, a cada cabeza se le asigna a una velocidad determinada. A la cabeza 1 (café) se le pone un movimiento senoidal que pareciera un círculo; a la cabeza 2 (blanco) la espiral de Arquímedes la cual se ira alejando del cuello conforme pase el tiempo para simular la espiral; la cabeza 3 (verde) será un movimiento del infinito de adelante hacia atrás; para la cabeza 4 (azul) un movimiento tipo medio círculo hacia la derecha y para la última cabeza (roja) será un movimiento vertical de arriba hacia abajo como si estuviera asintiendo.

Extra-5 puntos: Hacer que cada cabeza escupa/emita su aliento (por medio de texturas con traslucidez) al presionar una tecla:

- Roja: Fuego
- Azul: Rayos
- Verde: Gas Venenoso
- Blanco: Hielo
- Café: Arena

Bloque de código

```
// Para el aliento
#define TECLA_FUEGO GLFW_KEY_1
#define TECLA_RAYOS GLFW_KEY_2
#define TECLA_GAS GLFW_KEY_3
#define TECLA_HIELO GLFW_KEY_4
#define TECLA_ARENA GLFW_KEY_5
```

En nuestro Window.h definimos las teclas que vamos a utilizar para presionar y que salga nuestro aliento, las cuales serán los números 1 para la cabeza roja, 2 para la cabeza azul, 3 para la cabeza verde, 4 para la cabeza blanca y 5 para la cabeza café.

```
// Texturas para los alientos
Texture texturaFuego;
Texture texturaRayos;
Texture texturaGas;
Texture texturaHielo;
Texture texturaArena;

texturaFuego= Texture("Textures/Fuego.png");
texturaFuego.LoadTextureA();
texturaRayos = Texture("Textures/rayos.png");
texturaRayos.LoadTextureA();
texturaGas = Texture("Textures/gas_venenoso.png");
texturaGas.LoadTextureA();
texturaHielo = Texture("Textures/Hielo.png");
texturaHielo.LoadTextureA();
texturaArena = Texture("Textures/arena.png");
texturaArena.LoadTextureA();
```

Le pasamos la ubicación de nuestras texturas

```
//teclas para los alientos
alientoActivo[0] = mainWindow.getKeys()[TECLA_FUEGO];
alientoActivo[1] = mainWindow.getKeys()[TECLA_RAYOS];
alientoActivo[2] = mainWindow.getKeys()[TECLA_GAS];
alientoActivo[3] = mainWindow.getKeys()[TECLA_HIELO];
alientoActivo[4] = mainWindow.getKeys()[TECLA_ARENA];

// Actualizar tiempos de aliento
for (int i = 0; i < 5; i++) {
    if (alientoActivo[i]) tiempoAliento[i] += deltaTime;
    else tiempoAliento[i] = 0.0f;
}
```

Le pasamos las teclas a utilizar y el tiempo que va a estar utilizando

```
// Ubicacion inicial del aliento
const glm::vec3 OFFSETS_POSICION[5] = {
    glm::vec3(-2.0f, -2.0f, 1.05f), // Cabeza Cafe

    glm::vec3(-1.8f, -1.1f, 1.2f), // Cabeza Blanca

    glm::vec3(-1.8f, -1.5f, 0.45f), // Cabeza Verde

    glm::vec3(-1.8f, -1.1f, 0.03f), // Cabeza Azul

    glm::vec3(-1.8f, -1.6f, -0.01f) // Cabeza Roja
};

const float VELOCIDAD_ANIMACION = 0.01f;
const float ESCALA_ALIENTO_ANCHO = 0.3f;
const float ESCALA_ALIENTO_LARGO = 1.5f;
```

La ubicación inicial de cada aliento, para que este lo mas cercano a la boca

```

// Renderizado de alientos
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE);

for (int i = 0; i < 5; i++) {
    if (!alientoActivo[i] || tiempoAliento[i] <= 0.0f) continue;

    // 1. Posición base + offset
    glm::vec3 posicionAliento = glm::vec3(modelCabezas[i][3]) + OFFSETS_POSICION[i];
    glm::mat4 model = glm::translate(glm::mat4(1.0f), posicionAliento);

    // Rotación para la cabeza roja
    if (i == 4) { // Cabeza 5 (índice 4)
        // Rotación especial para el aliento 5
        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación X
        model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotación X
    }
    else {
        // Aplicar rotación de -90° en X a TODOS los alientos
        model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
    }

    // Rotación para orientar hacia adelante
    glm::vec3 direccion = glm::normalize(glm::vec3(modelCabezas[i] * glm::vec4(0.0f, 0.0f, 1.0f, 0.0f)));
    model = glm::rotate(model, atan2(direccion.x, direccion.z), glm::vec3(0.0f, 1.0f, 0.0f));

    // Escala
    float escalaVar = 1.0f + sin(tiempoAliento[i] * 2.0f) * 0.1f;
    model = glm::scale(model, glm::vec3(ESCALA_ALIENTO_ANCHO * escalaVar,
        ESCALA_ALIENTO_ANCHO * escalaVar,
        ESCALA_ALIENTO_LARGO));

    // Configurar textura
    switch (i) {
        case 0: texturaArena.UseTexture(); break;
        case 1: texturaHielo.UseTexture(); break;
        case 2: texturaGas.UseTexture(); break;
        case 3: texturaRayos.UseTexture(); break;
        case 4: texturaFuego.UseTexture(); break;
    }

    // Animación de textura más lenta
    glm::vec2 texOffset(0.0f, tiempoAliento[i] * VELOCIDAD_ANIMACION);
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(texOffset));

    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    meshList[4]->RenderMesh();
}

```

Configura y renderiza los efectos de aliento de las cinco cabezas de un dragón, activando primero la transparencia (GL_BLEND) para lograr un efecto translúcido. Para cada cabeza, calcula la posición del aliento sumando un offset predefinido a la posición de la cabeza, aplica rotaciones específicas y ajusta la orientación hacia adelante usando trigonometría (atan2). Luego, escala el aliento con una variación sinusoidal para darle dinamismo, asigna una textura diferente a cada tipo de aliento (arena, hielo, gas, rayos o fuego), y anima la textura desplazándola verticalmente para simular movimiento. Finalmente, renderiza el mesh del aliento y restaura la configuración inicial de profundidad y transparencia.

- **Problemas presentados.**

En esta practica presente problemas para poder mostrar el mensaje de PROYECYO ya que no logre que se mapeara las coordenadas individuales de cada letra y solo pude que se desplazara toda la textura que contenía la tipografía; modificando el mesh y pasarle las coordenadas, no lo pude lograr.

- **Conclusiones**

Estos ejercicios fueron de gran ayuda para aprender cómo funciona la animación por medio de código, como usar funciones matemáticas para poder lograr movimientos en nuestros objetos, el texto también fue un elemento clave para poder entenderlo. Sin duda me dejó un gran aprendizaje.

- **Bibliografía**

- ✚ (S/f-g). Klipartz.com. Recuperado el 26 de abril de 2025, de <https://c1.klipartz.com/pngpicture/132/647/sticker-png-cloud-thunder-lightning-blue-water-electric-blue-liquid-thumbnail.png>
- ✚ (S/f-h). Pngtree.com. Recuperado el 26 de abril de 2025, de https://png.pngtree.com/png-vector/20191120/ourmid/pngtree-brown-dusty-cloud-or-dry-sand-flying-with-wind-png-image_1992324.jpg
- ✚ (S/f-i). Pond5.com. Recuperado el 26 de abril de 2025, de https://images.pond5.com/green-poison-gas-toxic-icon-illustration-214428825_icon1.jpeg
- ✚ Microsoft Copilot. (2025). Imagen generada por inteligencia artificial del aliento [de hielo/fuego].